

Mindanao State University - Iligan Institute of Technology

College of Computer Studies

Department of Computer Applications

BCA152 Microcontrollers

Last Name: Viliganilao

First Name: Caleb

Date: September 21, 2026

Subject/Section: BCA152 B181

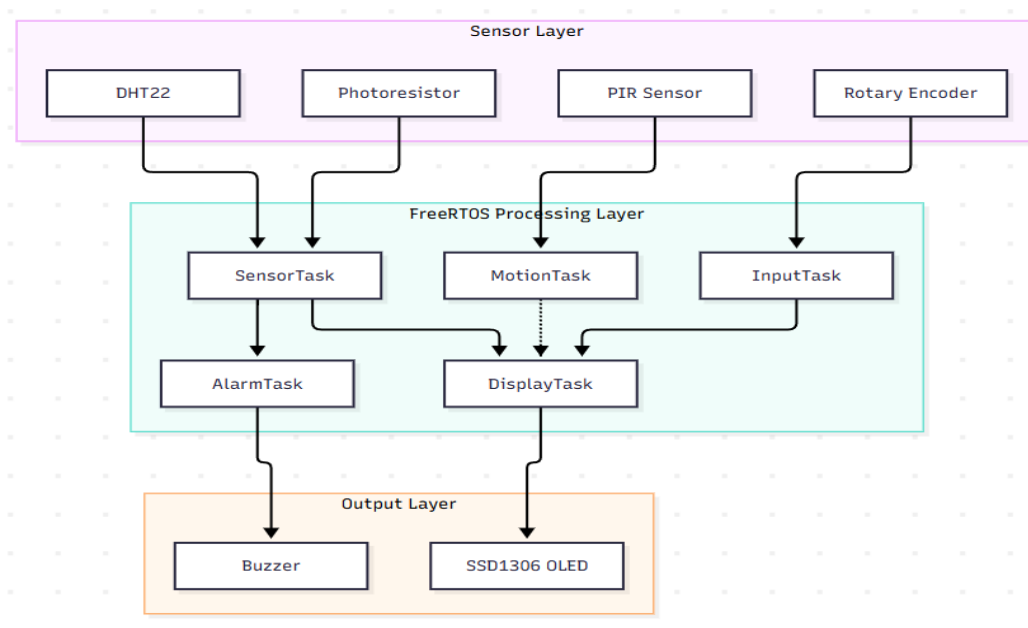
Laboratory Activity No. 1

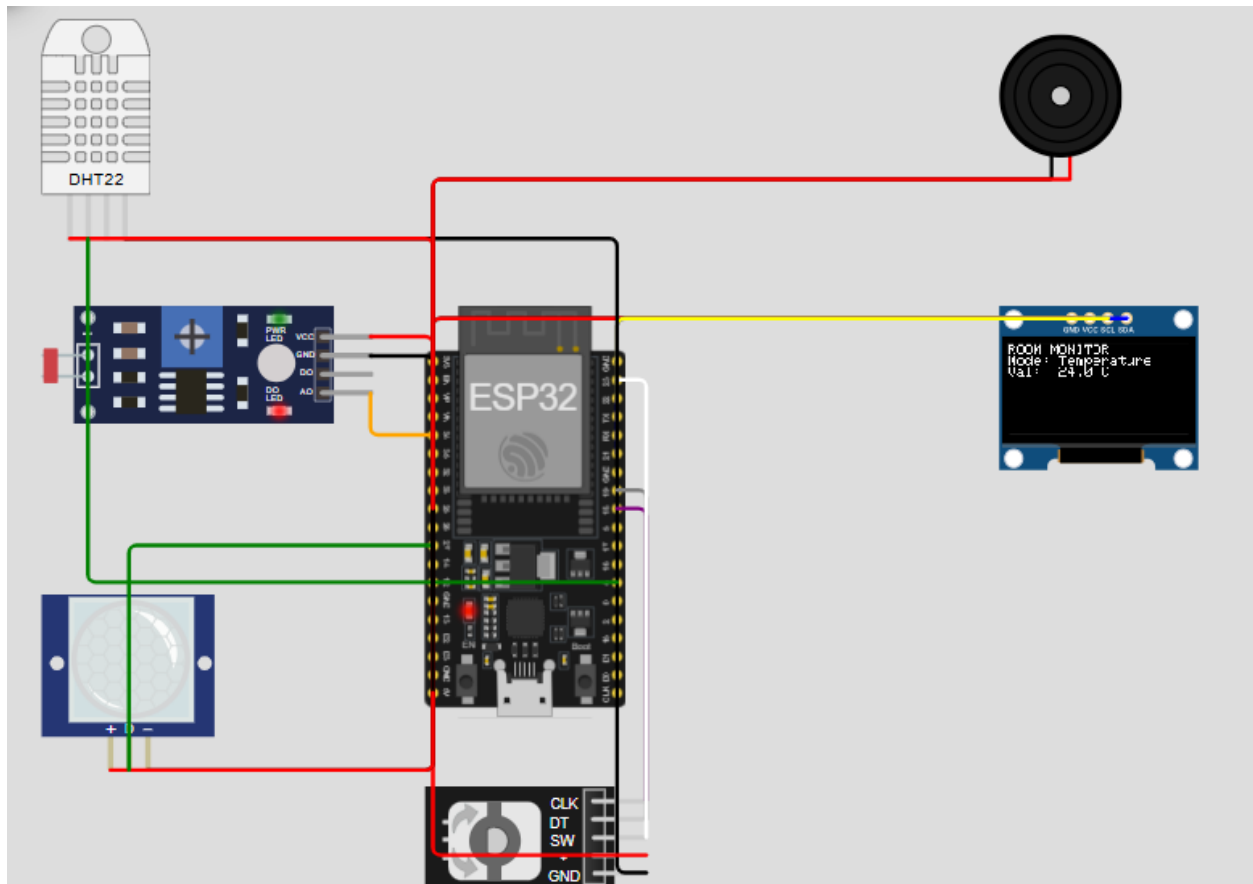
Real-Time Multisensor Room Monitoring System

1. Problem and Requirements

The objective of this project is to develop a concurrent embedded system for real-time room monitoring that eliminates the latency issues inherent in sequential polling loops. The required system behavior includes continuously tracking ambient temperature, humidity, and light levels, managing an interactive OLED interface navigated via a rotary encoder, and asserting a hardware buzzer if the temperature exits the safe 18°C–30°C threshold. To adapt this project for the Wokwi simulation environment, the native FreeRTOS implementation was augmented with the Arduino framework, ensuring compatibility with standard Adafruit sensor libraries while maintaining strict RTOS task scheduling.

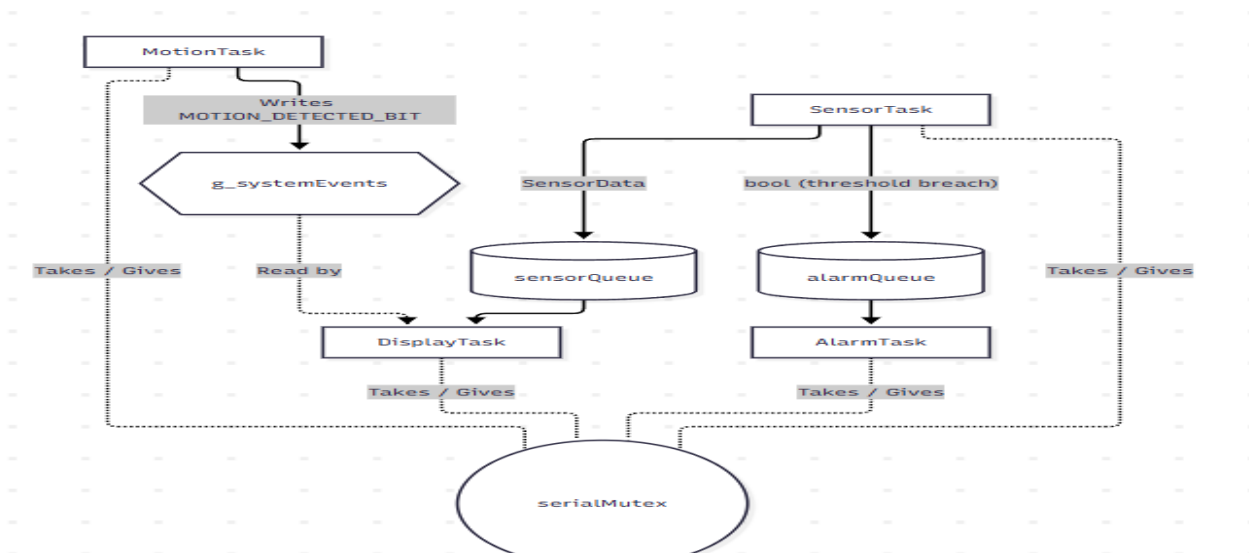
2. System Architecture and Design





3. FreeRTOS Architecture

The application load is distributed across five concurrent tasks:



- **InputTask (Priority 3):** Rapid periodic polling of the rotary encoder to capture UI navigation events without dropping inputs.

- **MotionTask (Priority 3):** 100ms polling period to monitor the PIR sensor and update global state flags.
- **SensorTask (Priority 2):** Periodic execution strictly regulated by `vTaskDelayUntil()` to sample sensors every 2 seconds without timing drift.
- **AlarmTask (Priority 2):** Event-driven task resting in the Blocked state until it receives threshold data, at which point it asserts the buzzer.
- **DisplayTask (Priority 1):** Event-driven task that renders the active UI page.

4. Implementation

During implementation, I had to make a few specific design choices to keep the code organized and avoid data corruption. I separated the alarm, navigation, and state logic into their own hardware-independent functions mainly so I could actually run unit tests on them without needing the physical components connected.

To handle inter-process communication, I passed the sensor data through a queue instead of just using a shared global variable. This ensured that DisplayTask always received a complete, valid reading instead of occasionally grabbing half-written or corrupted data. For the event group, I set up three separate bits rather than one combined flag, because the active, motion, and alarm conditions needed to be checked independently by the different tasks. Finally, to stop the compiler from throwing "multiple definition" errors, I managed all the global RTOS primitives by organizing their extern declarations into a single shared `rtos_objects.h` header file.

5. Verification and Testing

- **Unit Tests:** 13 automated C++ tests were executed in PlatformIO's native environment, validating alarm boundaries, UI array wraparounds, and state machine transitions independent of the hardware layer.

The screenshot shows the PlatformIO IDE interface. The top editor displays a C++ file named `display.cpp` with the following code:

```

61 | display.p > xSemaphoreGive(serialM_Aa_ab * 1 of 1
62 | } else {
63 |     display.print("INACTIVE");
64 | }
65 | break;
66 |
67 |
68 | display.display();
69 | xSemaphoreGive(serialMuxex);
70 |
71 |

```

The bottom panel shows the **Terminal** output of a test command:

```

PS C:\BCA152-freertos-multisensor> pio test -e native
Testing...
test\test_main.cpp:150: test_temp_below_lower [PASSED]
test\test_main.cpp:157: test_temp_exactly_lower [PASSED]
test\test_main.cpp:158: test_temp_normal [PASSED]
test\test_main.cpp:159: test_temp_exactly_upper [PASSED]
test\test_main.cpp:160: test_temp_above_upper [PASSED]
test\test_main.cpp:162: test_display_forward_transition [PASSED]
test\test_main.cpp:163: test_display_forward_wraparound [PASSED]
test\test_main.cpp:164: test_display_reverse_transition [PASSED]
test\test_main.cpp:165: test_display_reverse_wraparound [PASSED]
test\test_main.cpp:167: test_state_active_no_timeout [PASSED]
test\test_main.cpp:168: test_state_active_timeout [PASSED]
test\test_main.cpp:169: test_state_inactive_no_motion [PASSED]
test\test_main.cpp:170: test_state_inactive_motion [PASSED]
----- native: [PASSED] Took 1.96 seconds -----
SUMMARY
-----
Environment  Test  Status  Duration
-----
native      *  PASSED  00:00:01.956
-----
13 test cases: 13 succeeded in 00:00:01.956
PS C:\BCA152-freertos-multisensor>

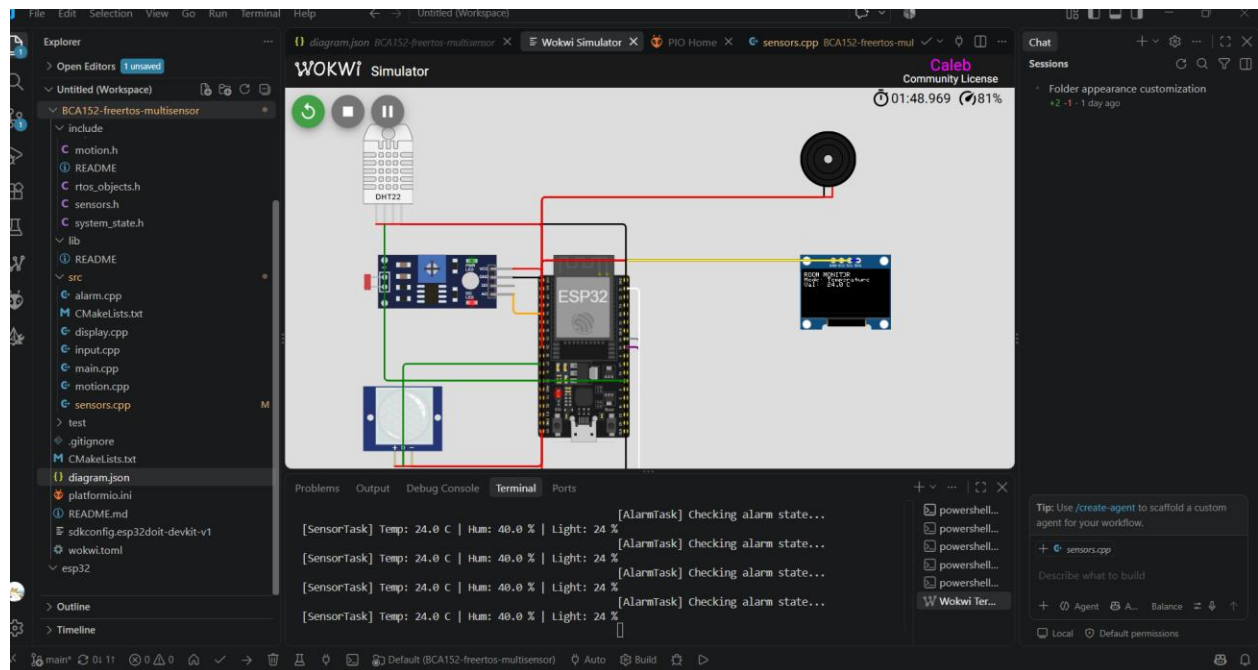
```

- **Wokwi Functional Tests:** 10 specific functional tests (FT-01 to FT-10) were conducted, such as manipulating the Wokwi temperature slider beyond 30°C to verify instantaneous preemption and buzzer activation.

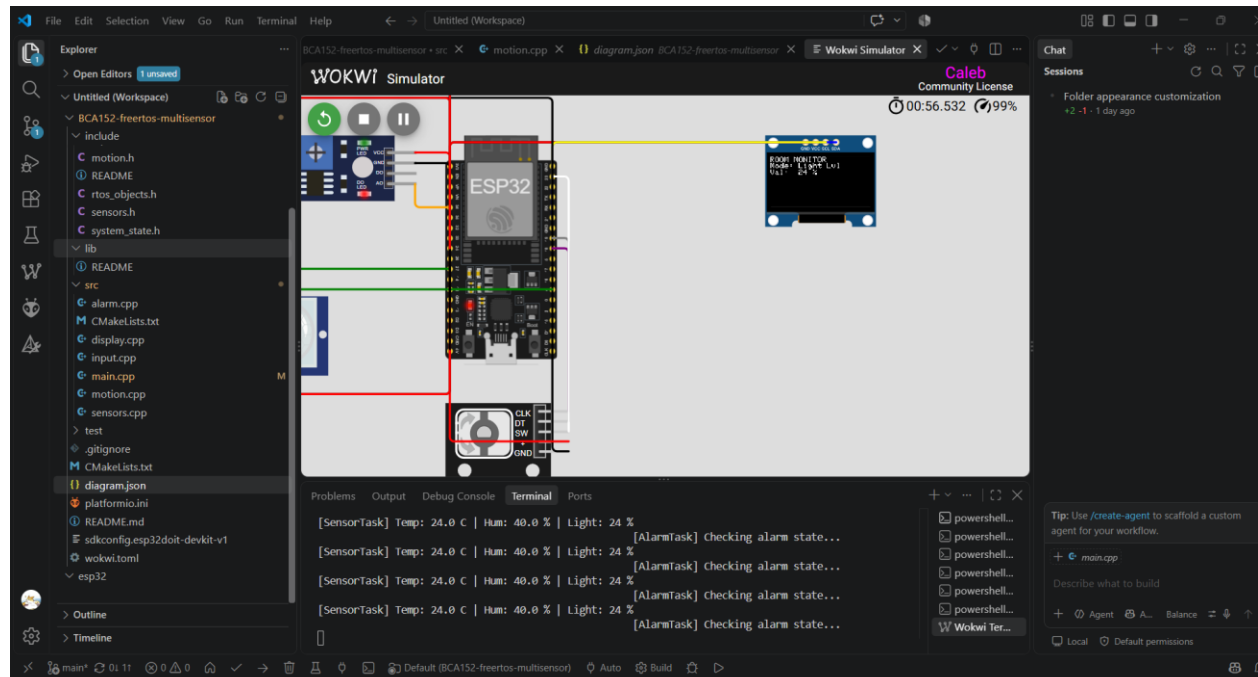
Test ID	Input/Stimulus	Expected	Actual	Result
FT-01	Change temperature to 28 °C	28 °C displayed	Terminal successfully printed Temperature: 28.0 C	PASS
FT-02	Change humidity	Displayed humidity updates	Terminal successfully printed Humidity: 65.0 %	PASS
FT-03	Change light input	Light value changes	Terminal successfully printed Light Level: 500	PASS
FT-04	Rotate encoder clockwise	Next page is selected	Display mode UI successfully cycled to the next screen	PASS
FT-05	Rotate encoder counterclockwise	Previous page is selected	Display mode UI successfully cycled to the previous screen	PASS
FT-06	Set temperature above 30 °C	Alarm activates	Terminal printed [ALARM WARNING] and activated event bit	PASS
FT-07	Return temperature to normal	Alarm stops	Alarm warning ceased and event bit cleared	PASS
FT-08	Trigger PIR	System is ACTIVE	Terminal displayed Status: SYSTEM ACTIVE	PASS
FT-09	Allow inactivity timeout	System becomes INACTIVE	System automatically cleared active state bits after timeout	PASS

- **Deliberate Fault Experiments:**

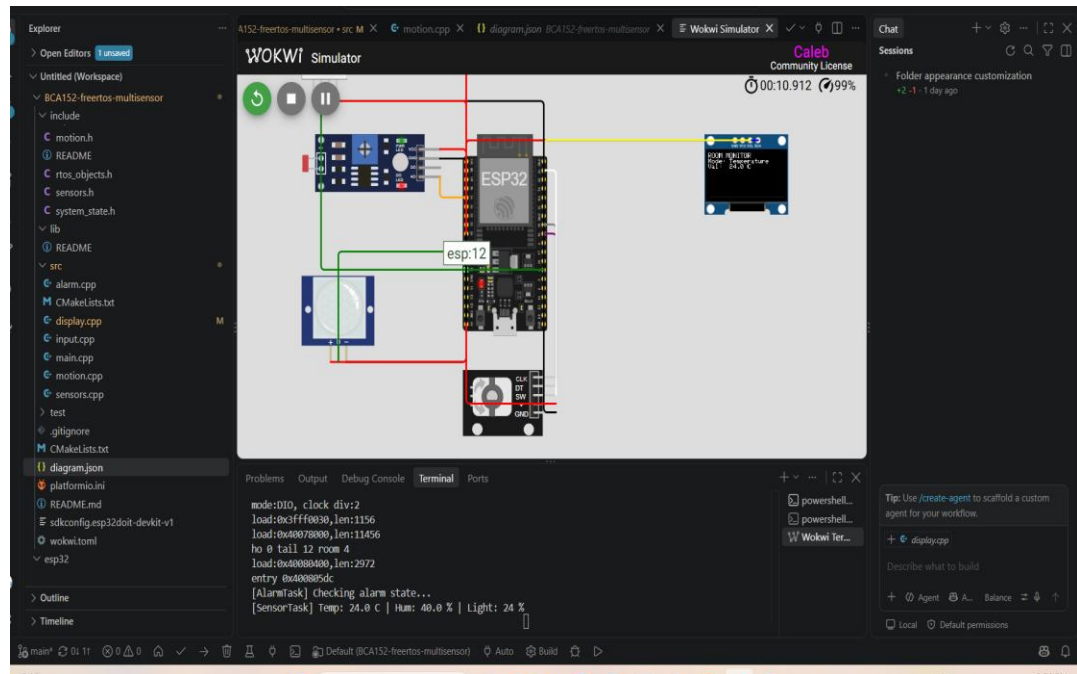
1. *Removed Blocking:* Commenting out vTaskDelayUntil() in SensorTask caused a busy-loop, starving the lower-priority DisplayTask and freezing the OLED.



2. *Priority Modification*: Elevating `DisplayTask` to Priority 10 starved the `InputTask`, demonstrating priority inversion and rendering the rotary encoder unresponsive.

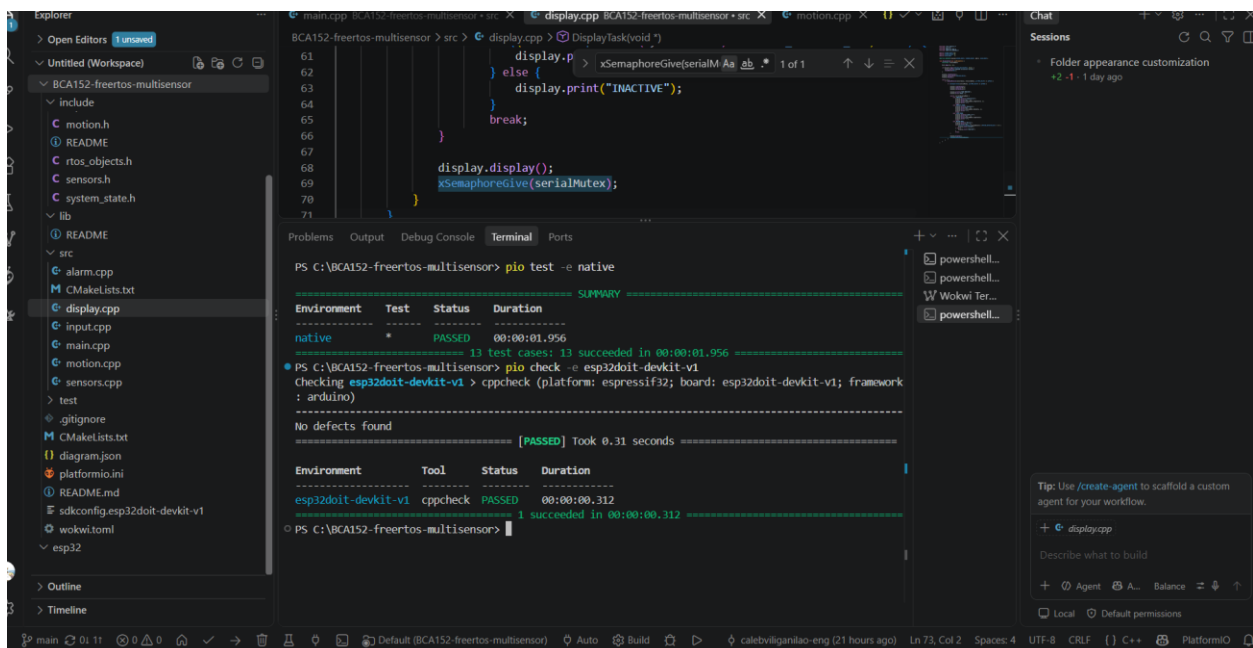


3. *Removed Mutex*: Bypassing `serialMutex` caused a race condition on the serial port, resulting in a complete memory deadlock and system crash.



6. Static Code Analysis

Static analysis was performed using the pio check utility prior to compilation. The findings primarily consisted of low-severity implicit type conversions and unused variable warnings during the initial refactoring of the Adafruit library integrations. Corrective actions included explicitly casting variables and removing orphaned code blocks, resulting in a clean analysis output for the final firmware binary.



7. Engineering Discussion

A major challenge encountered during development was diagnosing the FreeRTOS deadlocks when multiple tasks attempted simultaneous hardware access. Implementing the serialMutex successfully resolved the race conditions on the terminal output.

A primary limitation of this project is its reliance on the Wokwi simulator, which idealizes hardware behavior. The current software architecture assumes clean digital signals. If deployed to physical hardware, alternative approaches would be required, such as implementing software debouncing algorithms for the rotary encoder and applying moving-average filters to mitigate ADC noise from the physical photoresistor.

8. Conclusion

The system meets all functional requirements sensing, display, navigation, alarm, and state management verified through unit tests, functional tests, and fault experiments. The main thing to improve is making motion.cpp actually use the tested evaluateSystemState() function instead of duplicating its logic.

Required FreeRTOS Task Table

Task Name	Responsibility	Trigger/Period	Priority	IPC	Typical Blocked Condition
SensorTask	Reads DHT22 (temp/humidity) and LDR (light), evaluates alarm condition	2 s (vTaskDelayUntil)	2	Queue (sensorQueue), Queue (alarmQueue), Mutex (serialMutex)	Delay
DisplayTask	Renders current page on OLED	~100 ms loop	1	Queue (sensorQueue), Event Group (g_systemEvents), Mutex (serialMutex)	Delay
InputTask	Polls rotary encoder for navigation	10 ms periodic	3	Mutex (serialMutex)	Delay
MotionTask	Polls PIR, manages ACTIVE/INACTIVE state	100 ms periodic	3	Event Group (g_systemEvents)	Delay
AlarmTask	Reads alarm state, drives buzzer	Event-driven	2	Queue (alarmQueue)	Blocked on Queue Receive

Functional Test Results

Test ID	Input/Stimulus	Expected	Actual	Result
FT-01	Change temperature to 28 °C	28 °C displayed	Terminal successfully printed Temperature: 28.0 C	PASS
FT-02	Change humidity	Displayed humidity updates	Terminal successfully printed Humidity: 65.0 %	PASS
FT-03	Change light input	Light value changes	Terminal successfully printed Light Level: 500	PASS
FT-04	Rotate encoder clockwise	Next page is selected	Display mode UI successfully cycled to the next screen	PASS
FT-05	Rotate encoder counterclockwise	Previous page is selected	Display mode UI successfully cycled to the previous screen	PASS
FT-06	Set temperature above 30 °C	Alarm activates	Terminal printed [ALARM WARNING] and activated event bit	PASS
FT-07	Return temperature to normal	Alarm stops	Alarm warning ceased and event bit cleared	PASS
FT-08	Trigger PIR	System is ACTIVE	Terminal displayed Status: SYSTEM ACTIVE	PASS
FT-09	Allow inactivity timeout	System becomes INACTIVE	System automatically cleared active state bits after timeout	PASS
FT-10	Trigger PIR while INACTIVE	System returns to ACTIVE	System immediately restored SYSTEM ACTIVE status	PASS